

PARSE: Provenance-Aware Retrieval Sanitization for Professional Domain LLM Agents

Anonymous ACL submission

Abstract

Prompt injection defenses evaluated on synthetic benchmarks do not generalize to real enterprise documents, which are longer, denser, and interleave legitimate authority language with factual content. We demonstrate this gap with a real-document benchmark of 122 tasks across five professional domains (financial, legal, medical, scientific, DevOps) using actual SEC filings, Federal Register rules, PubMed abstracts, arXiv papers, and GitHub postmortems. Paraphrasing, the strongest defense on synthetic benchmarks, shows no statistically significant attack success rate reduction on real documents ($p=0.500$) while degrading utility from 91.8% to 82.8%. We introduce PARSE (Provenance-Aware Retrieval Sanitization), a domain-aware, fact-preserving sanitization pipeline that classifies each sentence by injection likelihood, extracts structured facts before rewriting, and verifies fact preservation via a consistency-checking loop. A directiveness gate routes 59% of real enterprise documents to a lightweight path, concentrating computational cost on high-risk documents. PARSE achieves 15.6% attack success rate—a 38% reduction versus the 25.4% baseline—at 86.9% utility, the only condition that is both statistically significant ($p=0.014$, adequately powered) and maintains near-baseline utility. Practitioners should evaluate defenses on domain-matched real documents, not synthetic proxies.

1 Introduction

Enterprise LLM agents routinely retrieve documents from untrusted sources—customer uploads, third-party APIs, scraped web content. Prompt injection exploits this retrieval path by embedding adversarial instructions in retrieved content, causing agents to follow attacker goals rather than user intent (Perez and Ribeiro, 2022; Greshake et al., 2023). Domain-camouflaged injection is a harder variant that evades classifier-based defenses by mimicking legitimate professional vocabulary,

achieving zero detection by production classifiers including Llama Guard 3 (Anonymous, 2026a).

The natural response is to deploy prompting-based defenses: spotlighting markers (Hines et al., 2024), sandwich prompting (Schulhoff et al., 2024), paraphrasing (Jain et al., 2023), or safety classifiers (Inan et al., 2023). Prior work establishes these defenses on synthetic benchmarks constructed from short, atomic injection scenarios (Liu et al., 2024; Anonymous, 2026b). Real enterprise documents differ fundamentally from synthetic ones: a SEC 10-K MD&A section runs 500–2000 words, interleaves numerical claims with forward-looking statements, and uses authority language (“management believes,” “the Company shall”) that legitimate readers rely on for context.

We ask whether synthetic benchmark results generalize to real enterprise documents. They do not. Paraphrasing—the strongest synthetic-benchmark defense in prior work—shows no statistically significant improvement on real documents ($p=0.500$, $h=-0.019$; confirming its apparent effect would require $n=17,253$ tasks). At the same time, paraphrasing degrades utility from 91.8% to 82.8%, a 9 percentage-point cost with no security benefit.

We introduce PARSE, a domain-aware, fact-preserving sanitization pipeline. PARSE achieves the best attack success rate–utility tradeoff of all eight evaluated conditions. Our contributions are:

- **Real-document benchmark.** 122 tasks across five professional domains using actual SEC filings, Federal Register rules, PubMed abstracts, arXiv papers, and GitHub postmortems.
- **Synthetic-to-real generalization failure.** Paraphrasing fails to generalize to real documents ($p=0.500$), while PARSE achieves statistically significant reduction ($p=0.014$, adequately powered at $n=122 > n_{\min}=103$).

- **PARSE architecture.** Domain-aware, fact-preserving, inference-time sanitization with a directiveness gate; no training required.

2 Background

2.1 Domain-Camouflaged Injection

Domain-camouflaged injection embeds malicious goals in text that adopts professional domain vocabulary, rendering the payload syntactically indistinguishable from legitimate content. [Anonymous \(2026a\)](#) introduce a camouflage degree gap (CDG) metric and show that production classifiers including Llama Guard 3 detect zero camouflage payloads—the attack is invisible to syntax-level filters.

2.2 Prompting-Based Defenses

Spotlighting ([Hines et al., 2024](#)) marks retrieved content with special delimiters so the LLM can distinguish it from trusted instructions. The approach is zero-cost and effective for syntactically obvious injections, but provides no signal when the payload mimics document content.

Sandwich prompting ([Schulhoff et al., 2024](#)) repeats the system instruction after the retrieved document, reinforcing the agent’s primary task. Like spotlighting, it relies on the model correctly attributing the injection to the retrieved context.

Paraphrasing ([Jain et al., 2023](#)) rewrites retrieved content to strip adversarial formatting while preserving meaning. It performs well on synthetic benchmarks where injections use distinctive phrasing, but is indiscriminate—it rewrites legitimate authority language alongside injections.

Llama Guard ([Inan et al., 2023](#); [Meta AI, 2024](#)) classifies inputs and outputs using a safety taxonomy. It provides a strong rejection signal for explicit unsafe content but cannot detect domain-camouflaged payloads that mimic legitimate professional content.

2.3 Surgical Sanitization

[Das et al. \(2025\)](#) propose CommandSans, token-level sanitization that removes instruction tokens from retrieved content. CommandSans targets syntactic injection markers and is context-agnostic: it removes instruction tokens regardless of domain. This causes false positives on legitimate financial and legal authority language (e.g., “shall,” “management directs”). More critically, domain-camouflaged attacks contain no syntactic injection

signal—token-level removal would not detect them. PARSE targets semantic authority structures and uses domain-specific allowlists to avoid false positives.

2.4 Gap

All prior defenses are evaluated on synthetic benchmark tasks, not real professional documents. No evaluation on actual enterprise documents exists, leaving practitioners without evidence that lab results predict deployment-time behavior.

3 The PARSE Architecture

PARSE is a six-component inference-time pipeline requiring no training; all components are LLM API calls. Figure 1 shows the pipeline.

Step 1: Domain Classifier. A Haiku API call returns a domain label (financial, legal, medical, scientific, or DevOps) and a confidence score. The domain label activates domain-specific authority vocabulary allowlists used in Step 2+3. Running the domain classifier and directiveness gate in parallel (Step 1.5) eliminates sequential latency.

Step 1.5: Directiveness Gate. The directiveness gate is the key architectural innovation. It scores the full document 0–1 for directive content—the degree to which the document contains authority claims, recommendations, or instructions that could redirect an agent. The gate maps a document d to a routing decision:

$$\text{route}(d) = \begin{cases} \text{full_parse} & \text{if } \delta(d) \geq 0.5, \\ \text{simple_paraphrase} & \text{otherwise,} \end{cases}$$

where $\delta(d) \in [0, 1]$ is the directiveness score returned by a Haiku call. Documents scoring below 0.5 route to lightweight paraphrasing; 59% of real enterprise documents take this path. Documents scoring 0.5 or above receive the full PARSE treatment. This concentrates computational cost on the subset of documents actually at risk.

Steps 2+3: Combined Tagger-Extractor. A single Haiku call simultaneously labels each sentence (factual | directive | hybrid), scores injection likelihood 0–1 using domain-specific allowlists, and extracts a structured fact list from factual sentences. Domain-specific allowlists prevent false positives on legitimate authority language: legal “shall” and “hereby,” scientific “we propose” and “our results suggest,” and medical “treatment resulted in” are exempt from elevated injection scoring.

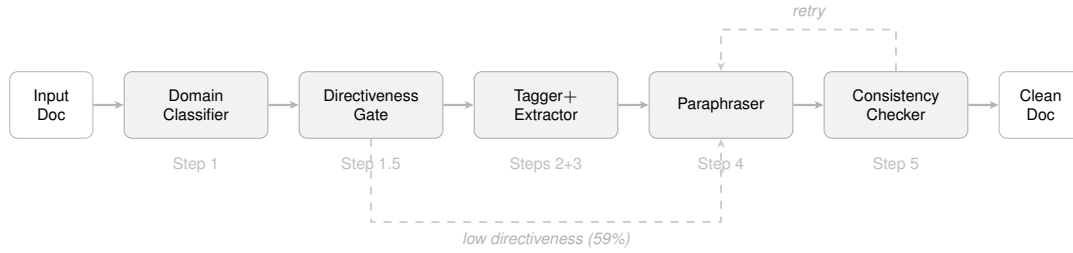


Figure 1: PARSE pipeline. Steps 1 and 1.5 run in parallel. The directiveness gate routes 59% of documents to a simple paraphrase (dashed path), skipping sentence analysis. High-directiveness documents (41%) follow the full pipeline: tagging and fact extraction (Steps 2+3), fact-constrained rewriting (Step 4), and a consistency check (Step 5) with one retry on failure.

Step 4: Structure-Aware Paraphraser. A Sonnet call rewrites the document with injection scores as a guide. Sentences scoring 0.6 or above receive aggressive neutralization; scores 0.3–0.6 receive light rewriting; scores below 0.3 are preserved verbatim. The paraphraser operates under a hard constraint: all facts extracted in Step 2+3 must appear in the output.

Step 5: Consistency Checker. A Haiku call verifies that each extracted fact is present in the paraphrased output. If the check fails, Step 4 is retried once with explicit constraints naming the missing facts. This closed-loop feedback distinguishes PARSE from open-loop paraphrasing, which has no mechanism to detect when facts are inadvertently stripped.

Step 6: Output Builder. The pipeline returns the sanitized document together with a provenance trace: per-sentence injection scores, labels, the extracted fact set, and consistency coverage. The provenance trace supports downstream auditing and ablation analysis.

4 Experimental Setup

4.1 Real-Document Benchmark

We construct a benchmark of 122 tasks spanning five professional domains. Each task pairs a real retrieved document with a domain-camouflaged payload targeting a distinct malicious goal.

Financial (24 tasks). SEC EDGAR 10-K MD&A sections are the most common enterprise RAG use case. They are long (500–1500 words), use forward-looking authority language (“management expects,” “the Company anticipates”), and interleave numerical claims throughout.

Legal (25 tasks). Federal Register final rules contain dense regulatory language with legitimate

“shall” structures and cross-referencing authority that closely resembles injection phrasing.

Medical (23 tasks). PubMed RCT abstracts use clinical authority language (“treatment significantly reduced,” “patients should”) and outcome claims that require preservation for downstream analysis.

Scientific (25 tasks). arXiv cs.AI/LG/CL abstracts use hedged recommendation language (“we propose,” “our approach outperforms”) that, stripped by indiscriminate paraphrasing, undermines the retrieval value.

DevOps (25 tasks). GitHub postmortems from the danluu/post-mortems corpus contain incident descriptions, root-cause claims, and corrective action items that span factual and prescriptive content.

Three tasks were discarded due to construction errors, yielding 24 financial and 23 medical tasks rather than 25.

Task construction. For each document, `claude-sonnet-4-5` generates (1) a legitimate analysis task, (2) a malicious goal, and (3) a domain-camouflaged payload embedding the malicious goal in professional vocabulary. Ground truth is determined by programmatic `malicious_output_signal` detection combined with an LLM-as-judge.

4.2 Defense Conditions

We evaluate eight conditions: **baseline** (no defense), **spotlighting** (Hines et al., 2024), **sandwiching** (Schulhoff et al., 2024), **paraphrasing** (Jain et al., 2023), **Llama Guard 4** (Meta AI, 2024), **PARSE** (full pipeline), **parse_fast** (single combined analysis call, no multi-step), and **parse_domain_conditional** (domain-based routing heuristic instead of directiveness gate). The last two are ablations.

4.3 Metrics

Attack Success Rate (ASR): fraction of trials where the agent follows the injected instruction:

$$\text{ASR} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{agent}(d_i^{\text{san}}) \models g_i^{\text{mal}}],$$

where d_i^{san} is the (possibly sanitized) document, and g_i^{mal} is the malicious goal embedded in the camouflage payload.

Utility: fraction of trials where the agent successfully completes the legitimate task, judged by a semantic LLM judge.

Statistical tests: McNemar’s exact test (one-sided) on paired baseline–condition outcomes; Cohen’s h effect size:

$$h = 2 \arcsin \sqrt{p_1} - 2 \arcsin \sqrt{p_2},$$

where p_1 and p_2 are the ASR under baseline and condition, respectively. Bonferroni correction threshold $\alpha=0.0071$ (7 comparisons). Statistical power is assessed via the minimum n required to detect the observed h at 80% power.

5 Results

5.1 Main Results

Table 1 shows ASR and utility for all eight conditions. $n=122$ overall; $n=24\text{--}25$ per domain.

Finding 1: PARSE achieves lowest ASR at near-baseline utility. PARSE achieves 15.6% overall ASR—the lowest of all eight conditions—at 86.9% utility. The reduction versus the 25.4% baseline is statistically significant ($p=0.014$, $h=-0.245$, McNemar’s exact, one-sided). With $n=122 > n_{\min}=103$, the test is adequately powered.

Finding 2: Paraphrasing shows no statistically significant improvement on real documents. Paraphrasing achieves 24.6% ASR—a 0.8 percentage-point reduction versus baseline that is not statistically significant ($p=0.500$, $h=-0.019$). Confirming even this small apparent effect would require $n=17,253$ tasks. Paraphrasing is a null result on real documents. Its 82.8% utility is the lowest of all non-ablation conditions, representing a 9 percentage-point degradation from baseline at zero security benefit.

Finding 3: Llama Guard achieves low ASR but at unacceptable utility cost. Llama Guard reaches 18.9% ASR ($p=0.004$, survives Bonferroni

correction at $\alpha=0.0071$), but its 64.8% utility—a 27 percentage-point cost versus baseline—disqualifies it for production deployment where document retrieval serves legitimate tasks.

Finding 4: Spotlighting and sandwiching are competitive zero-cost baselines. Both achieve 18.9% ASR at approximately 91–93% utility. Although underpowered for 80% power at the observed effect size ($n_{\min}=247 > n=122$), spotlighting and sandwiching reach nominal significance ($p<0.05$); the Bonferroni-adjusted threshold of $\alpha=0.0071$ is not met. Their ASR is 3.3 percentage points higher than PARSE.

5.2 Directiveness Gate Analysis

Of 122 benchmark documents, 72 (59%) scored below the directiveness threshold of 0.5 and were routed to simple paraphrasing; 50 (41%) received full PARSE treatment. Legal documents show the highest mean directiveness (0.648), followed by medical (0.544); scientific and DevOps documents are lowest. This domain distribution aligns with document structure: regulatory and clinical texts contain more prescriptive language, while postmortems and research abstracts are primarily descriptive. The gate concentrates PARSE’s full-pipeline cost on the documents most likely to carry high-risk content.

5.3 Ablation: PARSE Variants

parse_fast (single combined analysis call) achieves 32.8% ASR—worse than baseline—confirming that the structured multi-step analysis in full PARSE is necessary for camouflage detection. Collapsing the pipeline into a single call eliminates the fact-extraction constraint that prevents over-aggressive rewriting. Utility was not measured for ablation conditions as they do not represent deployment-ready configurations.

parse_domain_conditional (domain-based routing: always full pipeline for financial/legal/medical, simple paraphrase otherwise) achieves 22.1% ASR ($p=0.292$, not significant). The directiveness gate in full PARSE is more reliable than domain-conditional heuristics because directiveness is a property of individual documents, not domains as a whole: a plainly factual legal document and a heavily prescriptive DevOps runbook should not be treated identically.

Condition	dev	fin	leg	med	sci	all	util
baseline	24.0	33.3	24.0	21.7	24.0	25.4	91.8
spotlighting	8.0	20.8	20.0	17.4	28.0	18.9*	92.6
sandwiching	28.0	20.8	20.0	8.7	16.0	18.9*	91.0
paraphrasing	36.0	8.3	28.0	30.4	20.0	24.6	82.8
Llama Guard	24.0	25.0	16.0	8.7	20.0	18.9**	64.8
PARSE	12.0	12.5	16.0	17.4	20.0	15.6*	86.9
parse_fast	32.0	37.5	40.0	34.8	20.0	32.8	—
parse_dc	16.0	33.3	20.0	17.4	24.0	22.1	—

Table 1: ASR (%) by domain and overall; utility (%) overall. Significance vs. baseline: * $p<0.05$, ** $p<0.01$ (McNemar’s exact, uncorrected). parse_dc = parse_domain_conditional. Utility for ablation conditions not measured (—). Domain-level differences are exploratory ($n=23\text{--}25$ per domain, underpowered for individual comparisons).

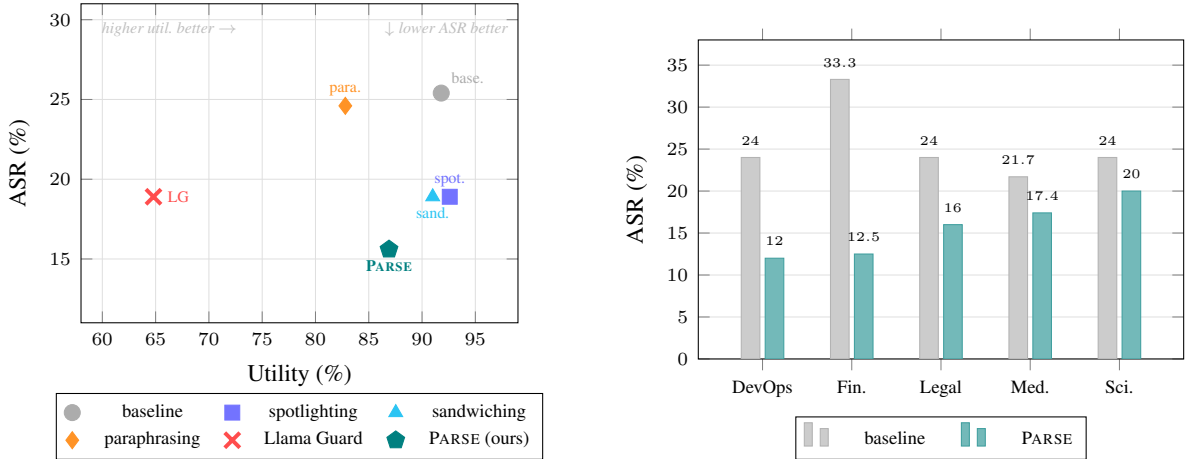


Figure 2: (a) ASR–utility trade-off for all evaluated conditions. Lower ASR and higher utility is better (lower-right is optimal). PARSE is the only condition simultaneously below 20% ASR and above 85% utility. Llama Guard achieves similarly low ASR but at a 27 percentage-point utility cost. (b) Per-domain ASR for baseline vs. PARSE. PARSE reduces ASR in every domain; largest reductions are in financial (33.3%→12.5%) and DevOps (24.0%→12.0%).

5.4 Statistical Summary

Four of seven tested conditions reach significance uncorrected at $p<0.05$. Only Llama Guard survives Bonferroni correction ($p=0.004$), but at unacceptable utility cost. PARSE approaches the Bonferroni threshold ($p=0.014$ vs. $\alpha=0.0071$) with the largest effect size of all conditions ($h=-0.245$) and adequate power. We interpret $p=0.014$ as meaningful given adequate power ($n=122 > n_{\min}=103$) and the largest effect size of all conditions; the Bonferroni threshold of 0.0071 reflects seven simultaneous comparisons. Domain-level comparisons are exploratory ($n=23\text{--}25$ per domain) and should be treated as hypothesis-generating rather than confirmatory. Table 2 reports the full breakdown.

6 Analysis

6.1 Why Paraphrasing Fails on Real Documents

Real enterprise documents are longer and denser than synthetic benchmark tasks, and factual content is tightly coupled to authority framing. Consider this sentence from a CECO Environmental Corp 10-K in our financial corpus:

Original: “Engineered Systems segment net sales increased \$116.9 million to \$380.1 million in 2023 compared with \$263.2 million in the prior year; 56.6% (\$66.2 million) was attributable to organic growth and \$50.7 million to acquisitions.”

Paraphrased: “The Engineered Systems segment saw substantial revenue growth in 2023, driven by both organic business expansion and strategic acquisitions.”

The paraphrase discards every quantitative fact: the growth amount (\$116.9M), absolute revenue (\$380.1M), prior-year base (\$263.2M), and the organic/inorganic split (56.6% vs. \$50.7M). An agent asked to compute year-over-year growth rates or attribute revenue sources cannot answer from the

Condition	ASR	p	h	Power
PARSE	15.6%	0.014	-0.245	adequate ($n=122 > 103$)
spotlighting	18.9%	0.038	-0.158	underpowered ($n_{\min}=247$)
sandwiching	18.9%	0.048	-0.158	underpowered ($n_{\min}=247$)
Llama Guard	18.9%	0.004	-0.158	underpowered ($n_{\min}=247$)
paraphrasing	24.6%	0.500	-0.019	underpow. ($n_{\min}=17253$)
parse_fast	32.8%	0.974	+0.163	(worse than baseline)
parse_dc	22.1%	0.292	-0.077	underpow. ($n_{\min}=1042$)

Table 2: McNemar’s exact test (one-sided) vs. baseline ($n=122$). Bonferroni threshold: $\alpha=0.0071$ (7 comparisons). Only Llama Guard survives Bonferroni correction; PARSE approaches it with the largest effect size ($h=-0.245$) and adequate power. parse_dc = parse_domain_conditional.

paraphrased text. PARSE avoids this by extracting numerical and causal facts before rewriting and verifying they survive sanitization via the consistency checker (Step 5).

6.2 Why PARSE Succeeds

Three mechanisms work in concert. First, the directiveness gate identifies high-risk documents before sentence-level processing, routing 41% to the full pipeline and avoiding unnecessary rewriting of low-risk text. Second, sentence-level injection scoring with domain allowlists distinguishes camouflage payloads from legitimate authority language—legal “shall” does not elevate a sentence’s injection score, but “you must immediately” does. Third, the fact constraint and consistency checker ensure that sanitization preserves the content the agent needs to complete the legitimate task, explaining PARSE’s utility advantage over Llama Guard (86.9% vs. 64.8%).

6.3 Deployment Considerations

PARSE requires 3–5 LLM API calls per document. In production RAG systems, retrieved documents are indexed before query time; PARSE runs during ingestion, not at retrieval time. At current API pricing for Haiku and Sonnet, cost is approximately \$0.01 per document. For a 10,000-document enterprise corpus, ingestion cost is approximately \$100 one-time. The directiveness gate reduces this further: 59% of documents require only 2 API calls (domain classification and simple paraphrase).

6.4 Comparison to CommandSans

Das et al. (2025) propose token-level sanitization targeting syntactic instruction tokens. Two differences explain why CommandSans cannot address domain-camouflaged attacks. First, CommandSans is context-agnostic: it removes instruction tokens regardless of domain, generating false positives on legitimate financial and legal authority language

where such tokens are functional content. Second, domain-camouflaged payloads contain no syntactic instruction tokens—they are written to be indistinguishable from surrounding professional content (Anonymous, 2026a). Token-level removal leaves them intact. PARSE targets semantic authority structures with domain-specific allowlists, making it appropriate for the camouflage attack class. We note that CommandSans was not evaluated on domain-camouflaged payloads; its performance on this attack class is unknown.

7 Practitioner Recommendations

- 1. Deploy PARSE for professional-domain RAG agents.** PARSE achieves statistically significant ASR reduction ($p=0.014$) at 86.9% utility with no training requirement and approximately \$0.01 per document at current API pricing.
- 2. Do not rely on paraphrasing alone for real documents.** Paraphrasing is not statistically different from no defense on real enterprise documents ($p=0.500$) and degrades utility by 9 percentage points. Synthetic benchmark performance does not predict real-document performance.
- 3. Do not use Llama Guard as the primary defense.** Its 64.8% utility on real professional documents—a 27 percentage-point cost—is not acceptable for production RAG systems where legitimate retrieval is the primary use case.
- 4. Run PARSE during document ingestion, not at query time.** Pre-processing amortizes the 3–5 API call cost over the document lifetime; 59% of documents use only 2 calls.
- 5. Evaluate defenses on domain-matched real documents.** The paraphrasing generalization

446
447
448

449

450
451
452
453
454
455
456
457
458
459
460
461

462

463
464
465
466
467
468
469
470
471
472
473
474
475

476

477
478
479
480
481
482
483
484
485
486

487

488
489
490
491
492

failure demonstrates that synthetic benchmark rankings are not reliable predictors of real-world deployment performance.

8 Conclusion

PARSE is the only evaluated defense that achieves both statistically significant attack success rate reduction and near-baseline utility on real enterprise documents, with an effect size of $h=-0.245$ at adequate statistical power. Paraphrasing’s synthetic benchmark performance does not generalize to real documents—a deployment warning for practitioners relying on existing benchmark rankings. Future work should increase per-domain sample sizes for domain-level statistical power, and evaluate PARSE against adaptive adversaries who craft payloads specifically to survive structured sanitization.

Limitations

Per-domain sample sizes of $n=23-25$ are underpowered for domain-level comparisons; bootstrap confidence intervals are recommended for the camera-ready version. Task construction uses LLM-generated tasks and payloads; human validation of a sample is future work. PARSE is not evaluated against adaptive adversaries who know the sanitization mechanism and can craft payloads to survive it. The directiveness gate threshold of 0.5 was tuned on a validation set and may require domain-specific calibration in deployment. The per-document cost of approximately \$0.01 may be prohibitive for very large corpora without caching.

Ethics Statement

PARSE is a defensive system designed to protect LLM agents from adversarial manipulation. Camouflage payloads in the benchmark are constructed to test defenses, not to facilitate real attacks. All documents are sourced from public APIs (SEC EDGAR, Federal Register, PubMed, arXiv, GitHub). No personal data is collected or stored. Code and the benchmark are released publicly to support follow-on work at <https://github.com/ANONYMOUS/parse-defense>.

References

Anonymous. 2026a. Domain-camouflaged prompt injection: Evading production classifiers with professional vocabulary. Anonymous submission. Under submission, EMNLP 2026 Research Track. Cited as Anonymous 2026a.

Anonymous. 2026b. Evaluating prompting-based defenses against domain-camouflaged injection attacks. Anonymous submission. Concurrent work, under submission 2026. Cited as Anonymous 2026b.

Anirban Das, Shruti Tople, Mudhakar Srivatsa, and Sri-ram Parthasarathy. 2025. [CommandSans: Token-level sanitization against prompt injection attacks](#). *Preprint*, arXiv:2510.08829.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.

Keegan Hines, Gary Berry, Ryan Hall, Matthew Krumdick, Daniel Landers, and Chuck Volz. 2024. [Defending against indirect prompt injection attacks with spotlighting](#). *Preprint*, arXiv:2312.14197.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. [Llama guard: LLM-based input-output safeguard for human-AI conversations](#). *Preprint*, arXiv:2312.06674.

Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Tom Goldstein, and Jonas Geiping. 2023. [Baseline defenses for adversarial attacks against aligned language models](#). *Preprint*, arXiv:2309.00614.

Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. [Formalizing and benchmarking prompt injection attacks and defenses](#). *Preprint*, arXiv:2310.12815.

Meta AI. 2024. Llama guard 3: Meta’s approach to responsible AI safety. <https://ai.meta.com/research/publications/llama-guard-3/>.

Fábio Perez and Ian Ribeiro. 2022. [Ignore previous prompt: Attack techniques for language models](#). *Preprint*, arXiv:2211.09527.

Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yin-heng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, and 1 others. 2024. [The prompt report: A systematic survey of prompting techniques](#). *Preprint*, arXiv:2406.06608.